STAGE 8: Production & Expert Skills (CI/CD, Architecture, Publishing)

Flutter & Dart: From Zero to Expert — Stage Textbook 8 of 8

Goal: Structure apps the way professional teams do, automate your build pipeline, and ship to real app stores.
Estimated time: 4–6 weeks
Prerequisite: Stage 7 (Advanced Flutter)

8.1 Clean Architecture in Flutter

A common, scalable structure separates an app into three layers:

```
lib/
  data/         # API clients, database access, repositories
  domain/       # Business logic, models, use-cases — no Flutter imports
  presentation/ # Widgets, screens, state management
```

```dart
// domain/models/user.dart — pure Dart, no Flutter dependency
class User {
  final int id;
  final String name;
  User({required this.id, required this.name});
}

// data/user_repository.dart
abstract class UserRepository {
  Future<User> getUser(int id);
}

class ApiUserRepository implements UserRepository {
  @override
  Future<User> getUser(int id) async {
    // calls http package, parses JSON, returns a User
    throw UnimplementedError();
  }
}

// presentation/user_screen.dart — depends on the abstract repository, not the API directly
class UserScreen extends StatelessWidget {
  final UserRepository repository;
  const UserScreen({super.key, required this.repository});
  // ...
}
```

Why it matters: Depending on an abstract `UserRepository` (not the concrete `ApiUserRepository`) means you can swap in a fake repository during tests, or swap the data source (REST, GraphQL, local DB) later without touching your UI code.

8.2 Dependency Injection

```dart
// Simple manual DI via constructor injection
void main() {
  final repository = ApiUserRepository();
  runApp(MyApp(repository: repository));
}

// Or with a package like get_it for larger apps:
import 'package:get_it/get_it.dart';

final getIt = GetIt.instance;

void setupLocator() {
  getIt.registerLazySingleton<UserRepository>(() => ApiUserRepository());
}

// Anywhere in the app:
final repo = getIt<UserRepository>();
```

Why it matters: Dependency injection makes it trivial to substitute a fake/mock implementation in tests, and avoids scattering `ApiUserRepository()` constructor calls throughout the codebase.

8.3 CI/CD with GitHub Actions

`.github/workflows/flutter_ci.yml`:

```yaml
name: Flutter CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:

      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.x'

      - run: flutter pub get
      - run: flutter analyze
      - run: flutter test
      - run: flutter build apk --release
```

Key rule: A CI pipeline should at minimum run static analysis (`flutter analyze`) and your test suite (`flutter test`) on every push, catching regressions before they reach `main`.

8.4 Performance Optimization

```dart
// Use const constructors wherever possible — Flutter can skip rebuilding them entirely
const Text('Static label');

// Avoid rebuilding expensive subtrees: extract them into their own const/cheap widgets
class ExpensiveChart extends StatelessWidget {
  const ExpensiveChart({super.key});
  // ...
}

// Use ListView.builder / GridView.builder for long lists (Stage 3) — never build
// hundreds of widgets up front with ListView(children: [...])

// Profile with Flutter DevTools to find unnecessary rebuilds:
// flutter run --profile, then open DevTools' "Performance" and "Widget rebuild" tabs
```

Checklist before shipping:

- Use `const` constructors everywhere a widget's inputs never change.
- Avoid heavy work (JSON parsing, sorting) inside `build()` — do it once and cache the result.
- Use Flutter DevTools to check for unexpected widget rebuilds and jank (dropped frames).

8.5 Publishing to the App Stores

Android (Google Play):

```
flutter build appbundle --release
```

1. Generate a signing key (`keytool -genkey ...`) and configure it in `android/key.properties`.
2. Upload the resulting `.aab` file from `build/app/outputs/bundle/release/` to the Play Console.
3. Fill out the store listing (screenshots, description, privacy policy URL) and submit for review.

iOS (App Store):

```
flutter build ipa --release
```

1. Requires a paid Apple Developer account and Xcode for code signing.
2. Open the generated `.xcarchive` in Xcode, or use `xcrun altool`/Transporter to upload to App Store Connect.
3. Fill out App Store metadata and submit for review.

Key rule: Both stores require versioning (`version` and `build number` in `pubspec.yaml`) to be incremented on every release, and both have a review process that can take from hours to several days.

8.6 Hands-On Project: Production-Ready Release Candidate

Take any app you built in a previous stage (the Shopping Cart app from Stage 4 is a good candidate) and bring it to release quality:

1. Refactor it into `data/`, `domain/`, and `presentation/` folders, with an abstract repository interface.
2. Add a `get_it` (or manual) dependency injection setup so the repository can be swapped for a fake one in tests.
3. Write at least 3 tests (unit and/or widget) that exercise the refactored code.
4. Add a GitHub Actions workflow that runs `flutter analyze` and `flutter test` on every push.
5. Run `flutter build appbundle --release` (or `apk --release` if you don't have a signing key yet) and confirm it builds successfully.

This project is the capstone of the entire roadmap: it touches architecture, testing, automation, and the final build step every real Flutter app must pass through.

8.7 Stage 8 Test

Question 1: In a clean-architecture Flutter app, which layer should contain zero Flutter (`package:flutter/...`) imports, and why?

Question 2: What problem does dependency injection solve when it comes to testing?

Question 3: What two commands should, at minimum, run in a Flutter CI pipeline on every push?

Question 4: Why does adding `const` to a widget constructor improve performance?

Question 5: What Flutter command produces the Android App Bundle used for a Play Store release?

Passing score: Get all 5 correct. Congratulations — you've completed the roadmap from zero to expert.